

STAY DATA STAY

Secure Login & Invalid-Attempt Handling

Authentication • Error Handling • Defensive Input Validation

OBJECTIVE

This report documents the secure-coding techniques used in the Stay Data Stay Python application to control login attempts, reject invalid input, and prevent the program from continuing when required conditions are not met.

```

30
31
32
33
34
35
36
37
38
39 Enter to log in..."
40
41 credentials/log in
42 name='KW_Data'
43 word='StayData123!'
44
45
46 out('Please enter your username: ')
47 l=input(('Thank You ')+user_name+'! Please enter your password: ')
48
49 .ng for incorrect credentials
50 me !=correct_username or user_password !=correct_password:
51 r if one of the login credentials is incorrect
52 ie incorrect credentials have been provided. Please try again.')
53 :=input('Please enter your username: ')
54 :word=input(('Thank You ')+user_name+'! Please enter your password: ')
55
56 ie start message
57 ie " + user_name + " to Stay Data Stay, where your data stays with you and does not get left behind! Let's get started!")
58
59 : they are currently on vacation or staying somewhere
60
61 : input(
62 currently on a trip or hotel stay?\n\n"

```

Figure 1. Login credential verification and generic error handling (lines 41–57).

AUTHENTICATION CONTROLS DEMONSTRATED

- **Credential verification:** The program compares the entered username and password against the expected credentials before allowing the user to continue.
- **Generic failure response:** If either credential is incorrect, the application displays one generic message rather than identifying whether the username or password failed.
- **Controlled program flow:** A while loop keeps the user inside the authentication step until both credential checks are satisfied, preventing invalid credentials from reaching the protected program flow.

Why the generic message matters

A single authentication-failure message reveals less information to an unauthorized user. It avoids confirming whether a particular username exists or whether only the password was incorrect.

DEFENSIVE PROGRAMMING

Input Validation & Security Maturity

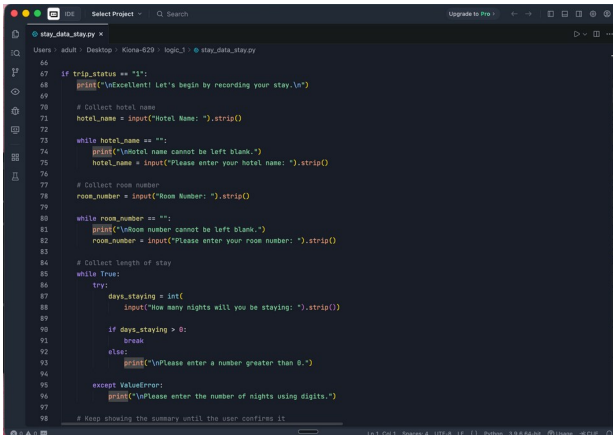


Figure 2. Blank-field, numeric, and range validation (lines 66–96).

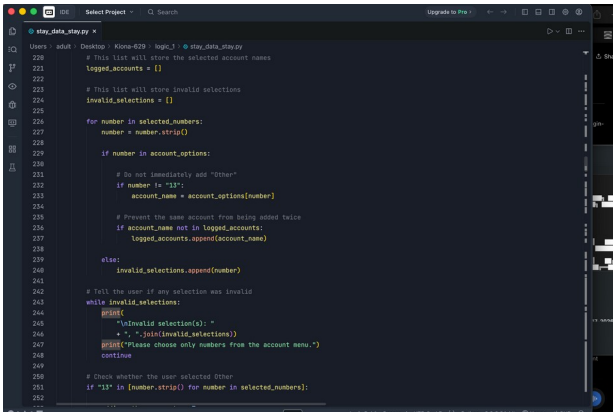


Figure 3. Approved-option checking and invalid-selection handling (lines 220–248).

ADDITIONAL CONTROLS DEMONSTRATED

Required-field validation Hotel name and room number cannot be blank. The program repeatedly prompts until required information is supplied.	Type & range validation The stay length is converted with int() inside try/except logic and must be greater than zero, preventing invalid numeric input from crashing or bypassing the flow.
Allow-list style checking Selected account numbers are compared against the application's existing account_options before they are accepted.	Invalid & duplicate handling Invalid selections are collected and rejected, while duplicate account entries are prevented before data is appended to the active list.

RUBRIC SUMMARY

Stay Data Stay applies defensive programming by validating authentication data and user-supplied values before allowing the program to progress. The implementation demonstrates credential checks, controlled retry behavior, exception handling, required-field validation, allow-list style selection checks, and protection against duplicate or invalid entries.

Security maturity note

This prototype currently stores demonstration credentials directly in the source code and accepts passwords through standard input. A production version should remove credentials from source control, use a trusted authentication service or protected database with salted password hashing, mask password entry, and add rate limiting or a maximum failed-attempt threshold to reduce brute-force risk.

Evidence prepared for rubric submission